



Technical Document

Last Revision: 19 September 2026 (Version 1.2)

Michael Koch

Author

EPI-USE Africa

Technical Services Internship Assessment

TABLE OF CONTENTS

Document information.....	1
1. Overview	2
2. Architecture	2
2.1 Architectural style	2
2.2 Layers inside the API	4
2.3 Data model	5
2.4 Keeping the hierarchy valid	6
2.5 Security	7
2.6 Deployment	7
3. Design patterns	8
4. Technologies and justification	8
4.1 Why Google Cloud Run.....	9
5. Additional functionality	9

Document information

Prepared for	EPI-USE Africa, Technical Services Internship Assessment
Author	Michael Koch
Version	1.2
Date	19 September 2026
Live application	https://ehm-351897716342.europe-west3.run.app
Source repository	https://github.com/MichaelKoch23/Epi-Use-Technical-Assessment
Demo credentials	Administrator <code>admin@example.com / Demo-Admin-2026!</code> ; read-only <code>viewer@example.com / Demo-Viewer-2026!</code>

The two demo accounts are seeded fixtures on a demo database and are defined in `api/app/seed.py`.

1. Overview

The system stores EPI-USE Africas employees and their reporting lines and lets HR staff view, search and change that structure. It is a React single-page application served by a Python API from one container on Google Cloud Run with all data in a managed PostgreSQL database.

What it does:

- **Brief requirements:** Create, read, update and delete employees, set and change reporting line managers, an interactive searchable org chart, a table view that sorts and filters on any field, Gravatar avatars.
- **Additional functionality:** Effective-dated reporting lines (view the organisation as at any date, schedule future changes), a costed preview before a branch is moved, role-based access that hides salaries from read-only users, a full audit trail, CSV and Excel import, CSV export, organisational analytics and photo upload. Section 5 describes each one.

Three decisions shape the design:

1. **PostgreSQL, with the hierarchy stored as a parent pointer on each employee.** The difficult part of this domain is not storing the tree but guaranteeing it can never contain a reporting loop, even when two people edit at the same time. A relational database with constraints, triggers and transactions can enforce that directly.
2. **A documented HTTP API as a deliverable in its own right.** The OpenAPI specification is generated from the same types that validate requests so the published contract always matches the code.
3. **One container image everywhere.** The image run locally is the image deployed so there is no separate production build that could behave differently.

2. Architecture

2.1 Architectural style

The solution is a client-server web application. The API is a **layered modular monolith**: one deployable service, split internally into modules (employees, hierarchy, auth, audit, analytics,

import and export) that talk through defined interfaces rather than over the network. The built React app is served by the same container so production has a single URL and no cross-origin requests.

Microservices were considered and rejected. The systems main rule, that the hierarchy never contains a loop, has to be checked in a single database transaction. Splitting the service would turn that into a distributed transaction problem for no benefit at this scale.

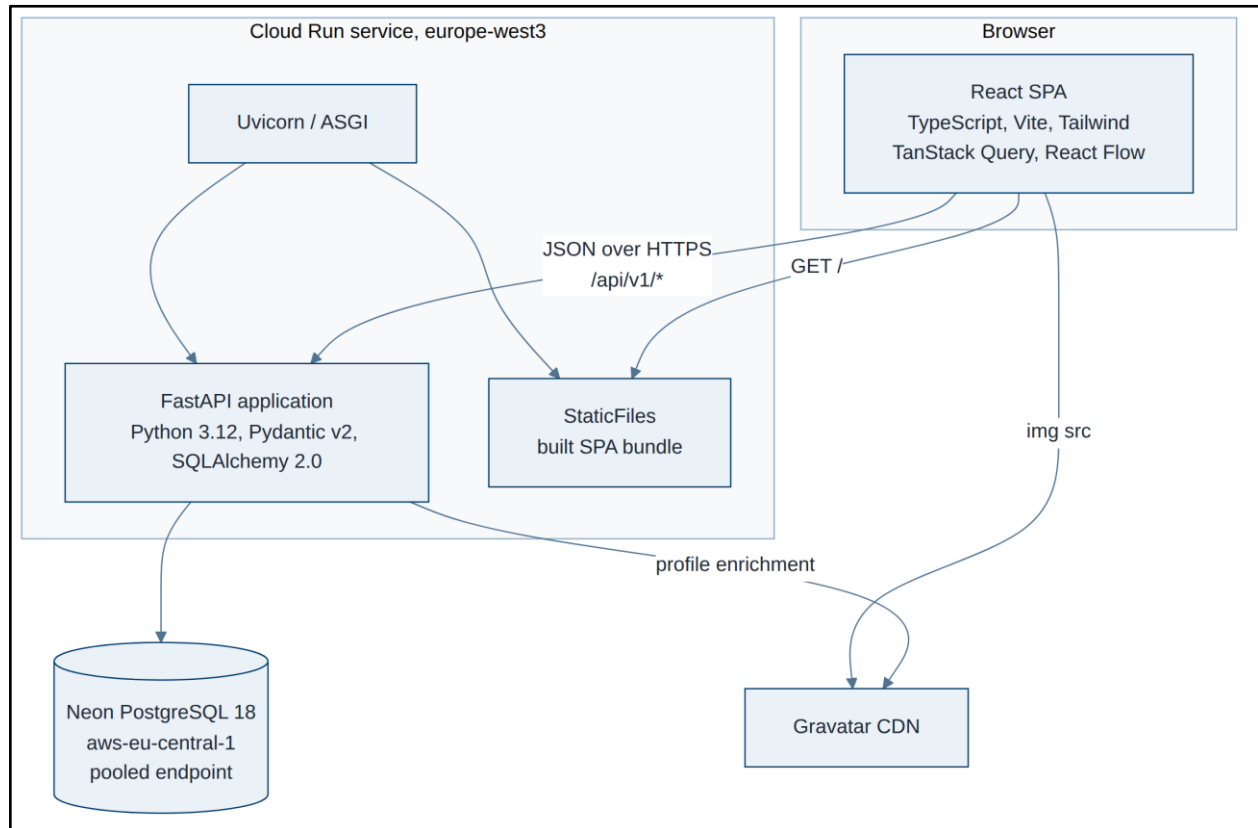


Figure 1: Container view: one Cloud Run service serving the React app and the API

2.2 Layers inside the API

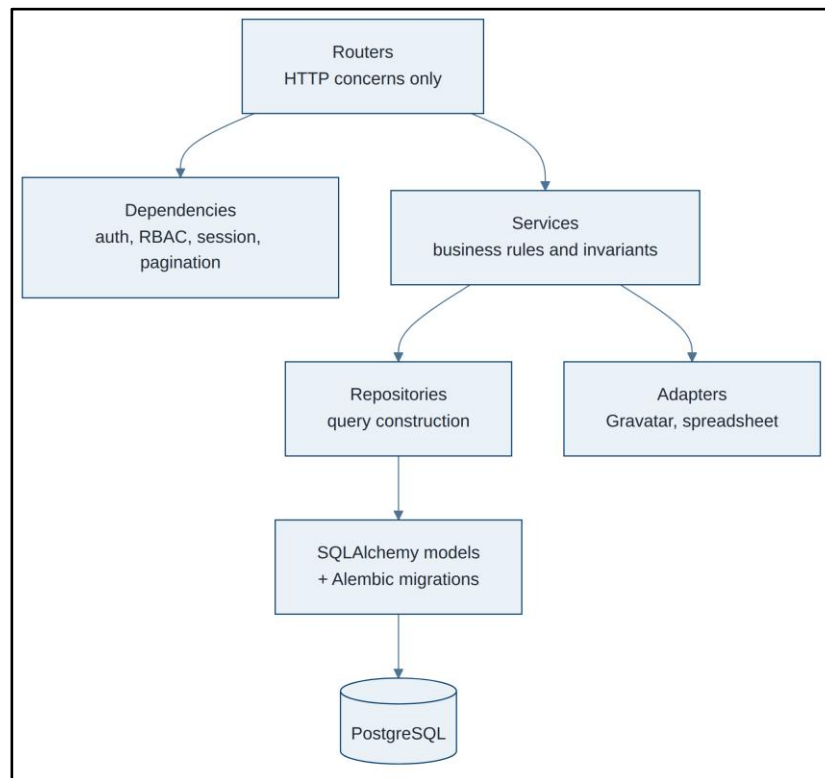


Figure 2: Layers inside the API

Layer	Responsibility	May not
Router	Parse HTTP input, choose the response shape, map errors to status codes	Contain business rules
Dependency	Authenticate, authorise, open the database transaction	Write data
Service	Enforce business rules, run transactions, write audit records	Know about HTTP
Repository	Build and run queries	Enforce business rules
Adapter	Wrap Gravatar and spreadsheet parsing	Leak external types upward

Dependencies only point downward so business rules can be tested without a web server and Gravatar can be replaced with a fake in tests.

2.3 Data model

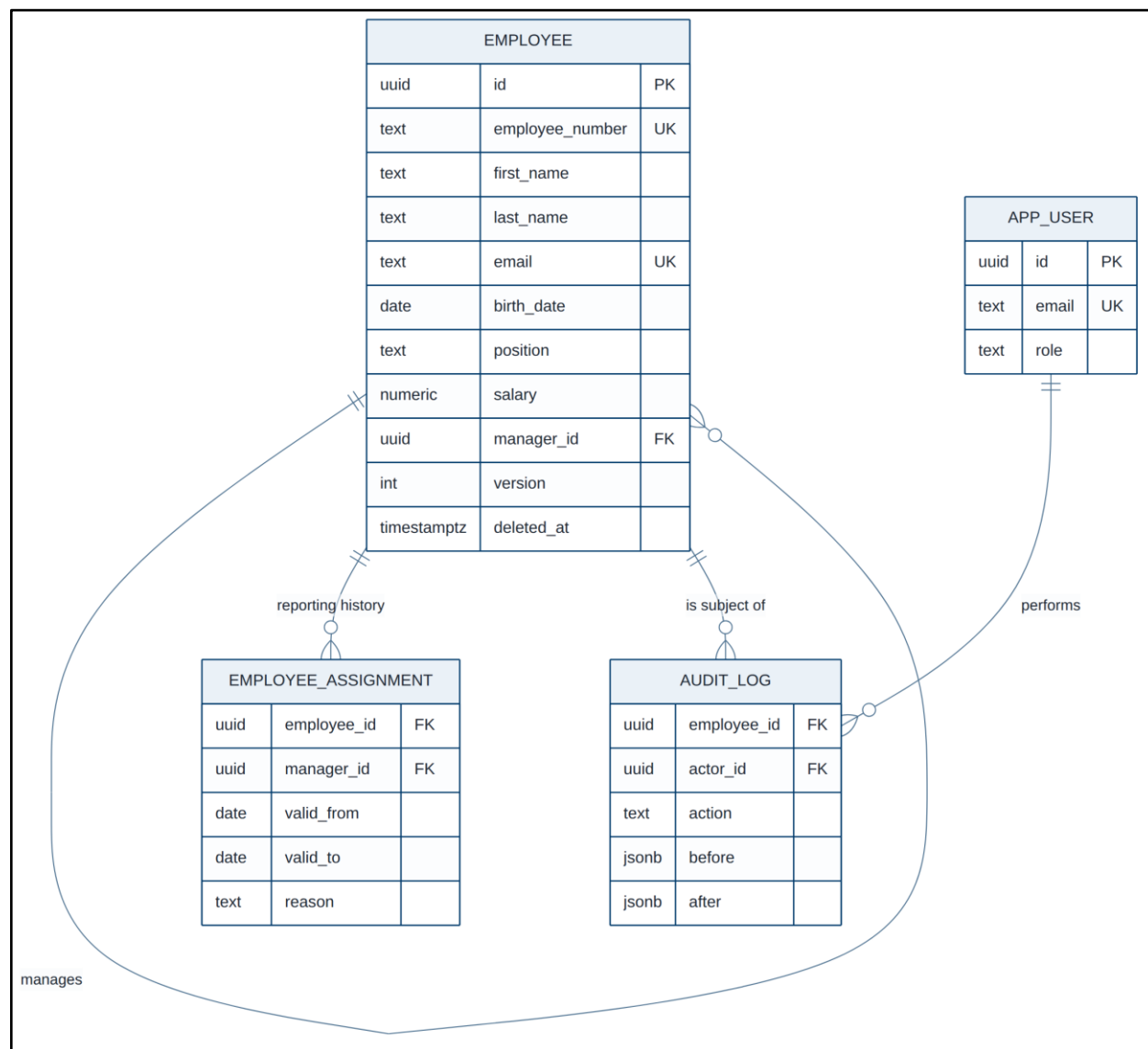


Figure 3: Main tables

Storing the hierarchy. Each employee row holds its manager's id and subtrees and reporting lines are read with recursive SQL queries. The alternatives (materialised paths, closure tables, a graph database) all read faster, but they store the structure a second time in a derived form that can fall out of step with the truth. At an organisation size of thousands of rows the recursive query takes a few milliseconds so the simpler model is the better trade. A graph database was rejected because the org chart is the simplest kind of graph (every employee has at most one manager), while the sortable, filterable table the brief asks for is exactly where relational databases are strongest.

Reporting lines over time. `employee_assignment` records every period during which an employee reported to a particular manager with a start date and an end date. A PostgreSQL exclusion constraint makes it impossible for an employee to have two managers on the same day. `employee.manager_id` is kept as a cached copy of today's manager, refreshed at the start of every write transaction so ordinary queries stay simple and a scheduled change takes effect on its date without any background job.

Soft deletion. Deleted employees are marked with `deleted_at` rather than removed, so history and the audit trail stay intact and a deletion can be undone.

2.4 Keeping the hierarchy valid

An employee may not be their own manager and indirectly neither (A reports to B, B to C, C to A). A check in application code alone is not enough: two simultaneous edits can each pass their own check and together create a loop and Cloud Run may run those edits in different instances. The rule is therefore enforced at four levels:

1. **Row locks:** A reassignment locks the rows involved before checking so conflicting moves run one after the other.
2. **A database trigger at commit:** The final structure is checked again when the transaction commits so a loop cannot be saved whatever the application does.
3. **Guarded queries:** Every recursive query stops if it revisits a row, so even corrupt data cannot cause an endless loop.
4. **Checks across future dates:** because changes can be scheduled a move is checked on its effective date and on every later date where another scheduled change takes effect. A move can be valid in September and still create a loop in October.

When an employee with direct reports is deleted the user chooses what happens to those reports: move them to the deleted employee's manager, leave them without a manager or delete the whole branch. The branch option always shows the affected people first. Concurrent edits to the same record are caught with a version number: a stale edit is rejected with a conflict response rather than silently overwriting someone else's change.

2.5 Security

- **Sign-in:** Passwords are hashed with Argon2id. Access tokens last 15 minutes. Refresh tokens last seven days are stored server-side and are replaced on every use, if an old refresh token is presented again every token in that chain is revoked, because it means the token has been copied.
- **Roles:** `hr_admin` can do everything, `viewer` can read but not change data. Every route declares its role and an automated test calls every route as an anonymous user with a forged token and as a viewer.
- **Salary confidentiality:** For a viewer the salary field is left out of every API response not hidden in the browser. Filtering or sorting by salary is also refused, since either could be used to work out individual figures.
- **Audit trail:** Every create, update, delete, restore and reassignment is written to the audit log in the same transaction as the change with before and after snapshots so the log cannot disagree with the data.
- **Input handling:** All requests are validated at the boundary, all queries use bound parameters, sort columns come from a fixed list, uploaded photos are re-encoded before storage and exported CSV values are escaped against spreadsheet formula injection.

2.6 Deployment

The container runs on **Google Cloud Run** in Frankfurt (`europa-west3`), and the database is **Neon PostgreSQL** in the same city. Both are in Frankfurt because Neon has no African region and a page makes several database round trips for each browser request so the application and database are placed together. Secrets are stored in Google Secret Manager. A Cloud Build trigger on the `main` branch builds the image and deploys a new revision, database migrations are applied as a separate step before the code that needs them. One instance is kept running so the assessor never waits for a cold start.

3. Design patterns

Pattern	Where	Why
Layered architecture	API structure	Keeps business rules separate from HTTP and the database
Repository	repositories/	Queries in one place, services can be tested against fakes
Service layer	services/	One place where business rules and transactions live
Unit of work	One transaction per request	A change and its audit record are saved together or not at all
Dependency injection	FastAPI Depends	Authentication, database sessions and permissions can be swapped in tests
Data transfer objects	Pydantic request and response models	Internal fields never leak, viewers and admins get different response shapes
Adapter	adapters/gravatar.py , adapters/spreadsheet.py	External services and file formats behind small, replaceable interfaces
Strategy	DeletionPolicy	Three deletion behaviours without branching logic in the service
Optimistic locking	version column and If-Match header	Concurrent edits are detected without holding locks while a user is editing
Specification (lightweight)	Composable list filters	Filters combine freely without a query method for every combination
Composite	Org chart nodes in the React app	A node and its whole subtree are handled the same way

4. Technologies and justification

Area	Choice	Why	Alternatives considered
Database	PostgreSQL 18 on Neon	Recursive queries, constraint triggers, exclusion constraints and exact decimals for salary, Neon's free tier does not expire	MySQL, MongoDB, Firestore, Neo4j
API	FastAPI (Python 3.12) with Pydantic v2	Validates requests and generates the OpenAPI specification from the same types	Express/NestJS, Spring Boot, Django REST, ASP.NET Core
Data access	SQLAlchemy 2.0 async with Alembic	Mature and typed and allows raw SQL for the recursive queries, migrations are versioned in git	Raw SQL, Prisma
Frontend	React 19, TypeScript, Vite	Strongest libraries for an interactive graph and a server-driven table, types generated from the API	Angular, Vue, Svelte

Area	Choice	Why	Alternatives considered
Styling	Tailwind CSS v4 and shadcn/ui	Accessible components kept in the repository and styled to the project	Material UI, Chakra
Org chart	React Flow with Dagre layout	Pan, zoom, minimap and drag-to-reassign built in, nodes are ordinary React components	D3, dedicated org-chart libraries
Data fetching	TanStack Query and TanStack Table	Caching and optimistic updates with automatic rollback, needed for drag-to-reassign, sorting and filtering done on the server	Redux Toolkit Query, SWR
Hosting	Google Cloud Run	Runs the delivered container unchanged, has no request time limit for large imports, and allows instant rollback	AWS, Oracle Cloud, Vercel, Render

4.1 Why Google Cloud Run

The deciding requirement was that an assessor opening the URL gets an immediate response, served by the same container that is in the repository.

- **AWS** would work well but needs a network, security groups, IAM, certificates and a load balancer, none of which is part of what was asked. Its free tier has also changed to an expiring credit balance.
- **Oracle Cloud** has a generous free tier, but capacity is often unavailable and its free database is Oracle rather than PostgreSQL, which would undo the data design above.
- **Vercel** is quick to set up but runs code as platform-specific functions rather than as the delivered container, with time limits that bulk import would have to work around.
- **Render's** free services go to sleep after fifteen minutes and take about a minute to wake up, which is not acceptable for a URL under assessment.

5. Additional functionality

Feature	What it does
Effective-dated reporting lines	View the org chart as it was on any past date or will be on a future one. Past dates are read-only so nobody edits history by accident
Scheduled changes	A reassignment can be dated in the future and takes effect on that day. Scheduled changes can be listed and cancelled
Structure comparison	Shows what changed in the reporting structure between any two dates

Feature	What it does
Costed branch move preview	Before a manager is moved, shows everyone who moves with them and for administrators, the salary cost moving between branches. Nothing is saved until the move is confirmed
Role-based access	Administrator and read-only roles with salaries removed from every response for read-only users
Audit trail	Every change is recorded with who made it, when and the values before and after, per employee and in a global feed
Import and export	CSV and Excel import with a dry run that validates the file without saving anything, CSV export of the filtered table
Analytics	Headcount, depth and span of control across the organisation, flagged structural anomalies, and cost roll-up per branch
Deletion policies	Choose what happens to an employee's reports when they are deleted, with a preview of who is affected
Photos and Gravatar	Upload a photo or fall back to Gravatar and then initials, Gravatar profiles can prefill the create form
Search	Command palette search by name, position or employee number that jumps to the person in the chart